ReadME for entire project:

```markdown
# Spatial Privacy Graph-Based Approaches for Location Privacy in IoT Smart Cities

## Overview

Smart cities rely on large-scale IoT deployments such as smartphones, connected vehicles, cameras, and environmental sensors. These devices continuously generate fine-grained spatial data that can unintentionally reveal sensitive user information such as home locations, workplaces, and detailed mobility patterns.

This repository presents **graph-based spatial privacy approaches** for protecting location privacy in IoT-enabled smart cities while preserving data utility for urban analytics, traffic management, and city-scale decision-making.

The project is implemented as a **completed, research-grade framework** that evaluates multiple spatial privacy-preserving algorithms under a **common simulation and evaluation pipeline**, enabling systematic and reproducible comparison of privacy-utility tradeoffs.

---
```

```markdown
## Objectives

- Protect user location privacy in smart city IoT systems
- Prevent re-identification from spatial and mobility data
- Preserve usefulness of location data for city-scale analytics
- Demonstrate privacy-utility tradeoffs using practical simulations
- Provide a comparative evaluation of spatial privacy mechanisms

---

## System Architecture

### Graph-Based Spatial Modeling

- Urban space is modeled as a **graph**
  - Nodes represent intersections or significant locations
  - Edges represent roads or connectivity
- User locations and movements are represented as paths over the graph
- Privacy regions are constructed using connected subgraphs or graph-constrained perturbations
```

Privacy Mechanisms

- **k-Anonymity**: Ensures every reported location is indistinguishable among at least _k_ users
- **Differential Privacy**: Adds calibrated noise to location data using formal privacy guarantees
- **Graph-Constrained Privacy**: Ensures anonymized locations respect urban topology
- **Density and Temporal Models**: Adapt privacy strength spatially and temporally

---

Implemented Approaches

This repository implements and evaluates **five spatial privacy-preserving algorithms**, all executed under identical datasets, metrics, and attacker assumptions.

Approach 1: Graph-Based k-Anonymity

Concept

- When a user shares a location, the system finds a connected subgraph containing at least _k_ users
- The exact location is replaced with the centroid of this region
- Preserves spatial realism while achieving anonymity

Key Metrics

- Location error
- Anonymization region size
- User coverage percentage

Approach 2: Differential Privacy on Location Data

Concept

- Adds Laplace noise to (x, y) coordinates before sharing
- Privacy controlled via the ε (epsilon) parameter
- Smaller ε provides stronger privacy with reduced accuracy

Approach 3: Graph-Constrained Differential Privacy

Concept

- Applies differential privacy while constraining noise to valid graph paths
- Prevents infeasible anonymized locations outside the city topology
- Improves utility compared to unconstrained coordinate perturbation

---

Approach 4: Density-Aware Adaptive k-Anonymity

Concept

- Dynamically adjusts _k_ based on local user density
- Reduces unnecessary anonymization in sparse regions
- Achieves a better privacy–utility balance than fixed-_k_ schemes

---

Approach 5: Trajectory Privacy via Temporal Cloaking

Concept

- Protects user mobility patterns across time
- Generalizes location updates within temporal windows
- Mitigates trajectory-based re-identification attacks

Privacy-Utility Tradeoff Analysis

All approaches demonstrate the fundamental tradeoff:

- **Higher privacy → Lower accuracy**
- **Lower privacy → Higher utility**

Evaluation includes:

- Original vs. anonymized locations
- Growth of anonymization regions
- Location error vs. _k_ / ε
- Noise distributions and trajectory distortion

````
---

## Repository Structure

```text
graph-based-location-privacy-iot/
│
├── README.md
├── requirements.txt
├── LICENSE
├── .gitignore
│
├── data/
│   ├── synthetic/
│   │   ├── city_graph.json
│   │   └── device_locations.csv
│   └── README.md
│
├── algorithms/
│   ├── k_anonymity/
│   ├── differential_privacy/
│   ├── graph_constrained_dp/
│   ├── density_aware_k_anonymity/
│   └── temporal_cloaking/
│
├── evaluation/
│   ├── metrics.py
│   ├── attacker_models.py
````
````

│   └── evaluator.py
│
├── experiments/
│   ├── run_all.py
│   └── configs/
│
├── results/
│   ├── plots/
│   ├── tables/
│   └── raw_json/
│
└── paper/
    ├── figures/
    ├── tables/
    └── notes.md
```

---

## Getting Started

### Installation

```bash
pip install -r requirements.txt
```

Running Individual Simulations
```

````markdown
### 1. Run Graph-Based k-Anonymity Simulation

Demonstrates spatial cloaking using graph-based k-anonymity.

```bash
python k_anonymity_simulation.py
```

### Outputs

- `figures/k_anonymity_demo.png`
- `figures/privacy_utility_analysis.png`
- `results/simulation_results.json`

## 2. Run Differential Privacy Simulation

Demonstrates location obfuscation using the Laplace mechanism.

```bash
python differential_privacy_simulation.py
```

### Outputs

- `figures/differential_privacy_demo.png`
````

- `figures/privacy_utility_analysis.png`
- `results/simulation_results.json`

3. Run Graph-Constrained Differential Privacy Simulation

```bash
python graph_constrained_dp_simulation.py
```

Outputs

- `figures/graph_constrained_dp_demo.png`
- `figures/privacy_utility_analysis.png`
- `results/simulation_results.json`

4. Run Density-Aware Adaptive k-Anonymity Simulation

```bash
python density_aware_k_anonymity_simulation.py
```

Outputs

- `figures/density_aware_k_anonymity_demo.png`
- `figures/privacy_utility_analysis.png`
- `results/simulation_results.json`

5. Run Temporal Cloaking Simulation

```bash
python temporal_cloaking_simulation.py
```

Outputs

- `figures/temporal_cloaking_demo.png`
- `figures/privacy_utility_analysis.png`
- `results/simulation_results.json`

Running Full Experimental Suite

All algorithms can be executed together using a unified runner:

```bash
python experiments/run_all.py
```

Current Limitations

- **Computational Complexity**
  Graph-based anonymization techniques can be computationally expensive when applied to large-scale graphs or high-density mobility datasets, limiting real-time applicability.

- **Privacy–Utility Trade-off**
  Stronger privacy guarantees (higher _k_ in k-anonymity or lower ε in differential privacy) inherently reduce spatial accuracy and utility of the released data.

- **Simplified Geographic and Mobility Constraints**
  Road networks, user movement patterns, and geographic constraints are abstracted for simulation purposes and may not fully capture real-world complexity.

- **Synthetic Datasets**
  Evaluations are conducted on synthetic datasets to ensure controlled experimentation, which may differ from real-world mobility behavior.

Future Directions

- **Edge and Fog-Based Privacy Computation**
  Offloading anonymization and privacy-preserving computations to edge and fog nodes to reduce latency and improve scalability.

- **Adaptive Graph Models for Real-Time Mobility**
  Dynamic graph construction and updating to reflect real-time changes in mobility patterns and network topology.

- **Hybrid Privacy Mechanisms**
  Combining k-anonymity, differential privacy, temporal cloaking, and graph constraints to achieve stronger and more flexible privacy guarantees.

- **AI-Driven Privacy-Utility Optimization**
  Leveraging machine learning and optimization techniques to automatically balance privacy and utility based on application requirements.

- **Integration with Real-World IoT Sensor Streams**
  Extending the framework to support live data from GPS-enabled devices, smart city sensors, and mobile IoT platforms.

Research Context

This project aligns with active research areas including:

- Spatial cloaking
- Graph-based anonymization
- Differential privacy
- Privacy-preserving IoT architectures for smart cities

The repository is designed as a **research-oriented and extensible foundation**, making it suitable for:

- Academic coursework and capstone projects
- Simulation-based experimentation
- Publication-quality empirical research

Research Contributions

- A **comparative framework** for evaluating multiple spatial privacy algorithms
- Implementations of **graph-based anonymization** and **differential privacy mechanisms**

```
- **Quantitative privacy–utility evaluation metrics** for systematic comparison
- **Clear visualizations** illustrating the effects of anonymization techniques
- A **modular and extensible codebase** to support further research and experimentation

---

## License

This project is licensed under the **Apache License 2.0**.

You may use, modify, and distribute this software in compliance with the license terms.
See the `LICENSE` file for full details.
```

ReadME for algorithms folder

```
# Algorithms – Graph-based Location Privacy for IoT

This directory contains the **core algorithmic implementations** used in the _Graph-based Location Privacy for IoT Smart Cities_ project.
```

```markdown
Each subfolder corresponds to a **distinct location
privacy mechanism or
privacy-preserving strategy**, implemented and
evaluated independently, while
following a **consistent documentation and
implementation structure**.

---

## Purpose of This Folder

The `algorithms/` directory serves to:

- Provide **modular implementations** of location
privacy algorithms
- Enable **fair comparison** between multiple
privacy mechanisms
- Support **experimental evaluation** within smart
city IoT simulations
- Act as a **research-ready codebase** for
extensions and benchmarking

Each algorithm is self-contained and documented
using a standardized README
format described below.

---
```

Folder Structure

```text
algorithms/
│
├── __init__.py
│
├── k_anonymity/
│   ├── k_anonymity.py
│   ├── README.md
│   └── config.yaml
│
├── differential_privacy/
│   ├── laplace_dp.py
│   ├── README.md
│   └── config.yaml
│
├── graph_constrained_dp/
│   ├── graph_constrained_dp.py
│   ├── README.md
│   └── config.yaml
│
├── density_aware_k_anonymity/
│   ├── density_aware_k.py
│   ├── README.md
│   └── config.yaml
│
└── temporal_cloaking/
```

    ├── temporal_cloaking.py
    ├── README.md
    └── config.yaml
```

Each subdirectory under `algorithms/` represents a self-contained
location privacy mechanism. Every algorithm follows a consistent structure:

- `*.py` — Core algorithm implementation
- `README.md` — Algorithm-specific documentation (theory, design, usage)
- `config.yaml` — Configurable parameters and experimental settings

This design enables modular experimentation, fair comparison, and
easy integration with the evaluation and experimentation pipeline.

Each algorithm resides in its **own subdirectory** with its own README.

---

## Standard README Structure (Per Algorithm)
```

```
Every algorithm subfolder follows the **same README structure** to ensure
clarity, reproducibility, and ease of comparison.

### 1. Algorithm Overview

- High-level description of the privacy mechanism
- Motivation and problem addressed
- Type of location privacy (e.g., differential privacy, graph-based, hybrid)

### 2. Theoretical Background

- Core privacy concepts used
- Mathematical or graph-based foundations
- Assumptions and threat model (if applicable)

### 3. Algorithm Design

- Step-by-step explanation of the algorithm
- Graph construction or spatial modeling approach
- Privacy parameter definitions (e.g., ε, k, budget)

### 4. Implementation Details

- Programming language used
- Key dependencies and libraries
```

```markdown
- Important functions or modules

### 5. Inputs and Outputs

- Expected input data format
- Privacy parameters and configuration options
- Output format and interpretation

### 6. Usage Instructions

- How to run the algorithm
- Example commands or function calls
- Configuration or parameter tuning notes

### 7. Privacy–Utility Considerations

- Impact of parameters on privacy and accuracy
- Known trade-offs
- Limitations of the approach

### 8. Evaluation Notes (If Applicable)

- Metrics used for evaluation
- Experimental assumptions
- Compatibility with the main evaluation pipeline

### 9. References
```

- Academic papers, standards, or prior work
- Links to relevant research

Integration with the Main Framework

Algorithms in this folder are designed to integrate with:

- Smart city IoT simulations
- Location privacy evaluation modules
- Visualization and analytics components
- Comparative privacy–utility analysis workflows

They can be executed independently or as part of a **larger experimental
pipeline** defined in the root project.

Conventions and Guidelines

- One algorithm per folder
- One `README.md` per algorithm
- Clear separation of logic and evaluation
- Consistent naming and parameter definitions
across algorithms

````markdown
- Research-oriented documentation style

---

## Credits

**Author:** Pushkal Gupta

**Project:** Graph-based Location Privacy for IoT
Smart Cities

---

## License

This project and all algorithms within this folder
are licensed under the
**Apache License, Version 2.0**.

You may not use this code except in compliance with
the License.
A copy of the License is available at:

```
http://www.apache.org/licenses/LICENSE-2.0
```
````